



Search and Hashing

CEN207 Data Structures — Week 6

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	Search costs, jump Anim 1–2 · interpolation Anim 3–4
2	Exponential Anim 5–6 · Fibonacci Anim 7–8 · hashing idea
3	Division Anim 9–10 · chaining Anim 11–12 · probing Anim 13–18 · rehashing Anim 19–20

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
Jump, interpolation, exponential, Fibonacci search	Sections 2–5
Hashing, division method, prime table size	Section 7
Collisions, chaining, load factor	Section 8
Open addressing, rehashing	Sections 9–10

How the code examples work

- Every idea has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-06/c/` and `code/week-06/java/`
- Each program's expected output is in the week notes

Recap — linear and binary search (Week 1)

- **Linear search:** check every cell, one by one — $O(n)$
- **Binary search:** sorted array, halve the range — $O(\log n)$
- Binary search needs the array **sorted** first
- Today: strategies between and beyond these two

Recap — arrays and linked lists (Week 2)

- **Array:** contiguous memory, $O(1)$ access by index
- **Linked list:** nodes and pointers, $O(1)$ insert at head
- Search structures below build directly on both
- A hash table's bucket is often a small linked list

Map of the week — at a glance

Faster search (sorted)	Hashing
Jump, interpolation search	Direct addressing, division method
Exponential, Fibonacci search	Chaining, open addressing, rehashing

1. The Search Problem and Costs

A question to start

Binary search already takes only $O(\log n)$.
Is there any reason to look for something
even faster, or something completely different?

What "search" means here

- Given a collection and a **target** key, find its location
- Or correctly report: the key is **not present**
- We count **comparisons** (or **probes**) as the cost
- Fewer comparisons for the same correctness = a better search

Cost, restated as a question

- Linear search: no assumption on the data, $O(n)$ worst case
- Binary search: needs **sorted** data, $O(\log n)$ worst case
- Can sorted data be searched even faster than $O(\log n)$?
- Can we search **without comparisons** at all?

When binary search isn't enough

- Binary search always checks the **middle**, no matter what
- But real data often has **structure** binary search ignores
- Evenly spread values? A formula can guess better than "middle"
- Data outside memory (disk, tape)? Every access has to count

Today's toolbox — five strategies

Strategy	Exploits
Jump search	Cheap block jumps beat many comparisons
Interpolation search	Values are roughly evenly spread
Exponential search	Target is likely near the front
Fibonacci search	No division/multiplication needed

Mini-quiz

Binary search always looks at the array's **middle** element first. Under what condition might a smarter first guess do better?

Answer

When values are roughly evenly spread.

The middle is only "index-optimal" — a value-aware guess (Section 3) can land closer to the target.

2. Jump Search

A question to start

Binary search jumps to the **middle** every time.
What if jumping in **fixed-size blocks** instead
turned out to be simpler, and still fast?

A short history

- Block-style search appears in Knuth's *Sorting and Searching* (1973), as an alternative to binary search
- The key design choice: how big should each block be?
- The answer, `block = floor(sqrt(n))`, balances two costs

Intuition — skipping pages in a book

- Flipping one page at a time to find a word: too slow
- Binary search: always split remaining pages in half
- Jump search: flip **whole sections** first, then one page
- Stop the big jumps once you have passed the word

The jump search idea

- Requires a **sorted** array, exactly like binary search
- `block = floor(sqrt(n))` : the jump distance
- Jump forward one block until a boundary is `>= target`
- Then scan that one block **linearly**, left to right

Why $\text{sqrt}(n)$?

- n/block jumps, then up to block linear steps
- Total cost: $n/\text{block} + \text{block}$ comparisons, worst case
- This sum is smallest when $\text{block} = \text{sqrt}(n)$
- Result: $O(\text{sqrt}(n))$ — between $O(\log n)$ and $O(n)$

Jump search, step by step

target = 42

block = $\text{floor}(\sqrt{16}) = 4$

comparisons: 6



Result: $\text{arr}[10] = 42$, found in 6 comparisons (3 jumps + a linear scan). $O(\sqrt{n})$.

Edge case — target in range but absent

target = 45

block = $\text{floor}(\text{sqrt}(16)) = 4$

comparisons: 7



Not found: 45 is not in the array. 7 comparisons in total — far fewer than linear search's $n = 16$: $O(\text{sqrt}(n))$.

Code — finding the right block

```
int block = (int) sqrt((double) n);
if (block < 1) block = 1;
int prev = 0, step = block, comp = 0;
while (step < n) {
    comp++;
    if (arr[step - 1] >= target) break;
    prev = step;
    step += block;
}
```

Code — linear scan inside the block

```
if (step > n) step = n;
for (int i = prev; i < step; i++) {
    comp++;
    if (arr[i] == target) return i;
    if (arr[i] > target) break;
}
return -1;
```

Complexity

- Block-finding phase: at most n/block jumps
- Linear-scan phase: at most block comparisons
- With $\text{block} = \sqrt{n}$: total $O(\sqrt{n})$
- Worse than binary search's $O(\log n)$, but simpler code

Common mistakes

- Forgetting the array must be **sorted** first
- Using a fixed block size instead of `sqrt(n)`
- Scanning past the block boundary instead of stopping there

Mini-quiz

An array has $n = 100$ elements. What block size does jump search use, and how many comparisons does it need in the worst case?

Answer

block = 10 ($\text{sqrt}(100)$). Worst case: about
10 jumps plus 10 linear steps — **~20**
comparisons, far below linear search's 100.

3. Interpolation Search

A question to start

A phone book is sorted **and** roughly evenly spread. Would you really start a search for "Smith" at the exact middle page?

A short history

- The idea appears in 1957, in early work on searching ordered files (W. W. Peterson)
- Later formalized and analyzed as **interpolation search**
- Best case on **uniform** data: close to $O(\log \log n)$

Intuition — a phone book

- Looking for "Smith": jump close to the back, not the middle
- Looking for "Baker": jump close to the front
- The jump uses the **value**, not just position, to guess
- Binary search only ever uses position — it ignores values

The interpolation search idea

- Requires a **sorted** array, exactly like binary search
- Estimate `target` 's position **proportionally** between ends
- `pos = lo + (target-arr[lo])*(hi-lo) / (arr[hi]-arr[lo])`
- Narrow `lo / hi` toward `pos` , exactly like binary search

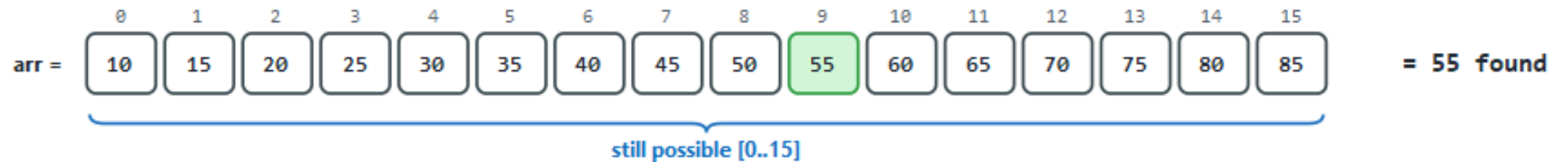
Interpolation search, step by step

target = 55

binary search here would take ~4 comparisons

probes: 1

$$\text{pos} = 0 + \text{floor}((55 - 10) * (15 - 0) / (85 - 10)) = 9$$



Result: arr[9] = 55, found in just 1 probe. On uniform data, interpolation search reaches the answer in far fewer steps than binary search's ~4 comparisons.

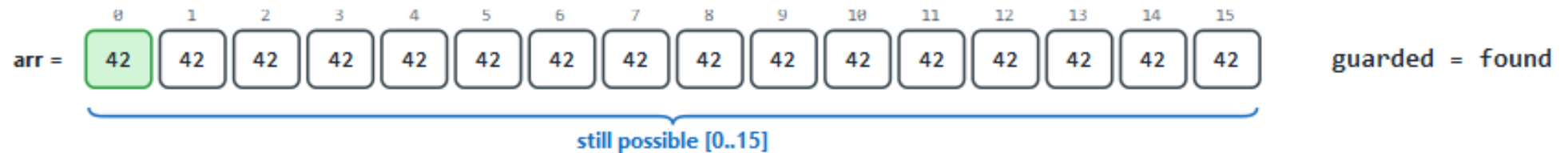
Edge case — a guard against division by zero

target = 42

binary search here would take ~4 comparisons

probes: 1

`arr[hi] == arr[lo] (42)` – cannot divide, the guard kicks in: this range is a single repeated value.



Result: `arr[0] = 42`, found in just 1 probe. On uniform data, interpolation search reaches the answer in far fewer steps than binary search's ~4 comparisons.

Code — the position formula

```
while (lo <= hi && target >= arr[lo]
      && target <= arr[hi]) {
    if (arr[hi] == arr[lo]) {
        /* ... guard: avoid division by zero ... */
        return lo;
    }
    int pos = lo + (int) ((double)
        (target - arr[lo]) * (hi - lo)
        / (arr[hi] - arr[lo]));
```

Code — narrowing the range

```
if (arr[pos] == target) return pos;  
if (arr[pos] < target) lo = pos + 1;  
else hi = pos - 1;
```

Complexity

- Uniform data: close to $O(\log \log n)$ on average
- Skewed data (one huge outlier): degrades toward $O(n)$
- Worst case is never better than binary search's $O(\log n)$
- The formula's power depends entirely on the data's shape

Common mistakes

- Forgetting the `arr[hi] == arr[lo]` division guard
- Using interpolation search on **skewed** (non-uniform) data
- Assuming $O(\log \log n)$ is a worst-case guarantee — it isn't

Mini-quiz

An array's values are extremely **skewed** — mostly small, one huge outlier at the end. Is interpolation search still a good choice?

Answer

No, not necessarily. Skewed data can push interpolation search's cost toward **$O(n)$** — plain binary search's $O(\log n)$ becomes the safer choice.

4. Exponential Search

A question to start

You are searching a huge, **sorted** stream of data, and you do not know its length. Where could binary search even start?

A short history

- **1976** — Jon Bentley and Andrew Yao publish "An almost optimal algorithm for unbounded searching"
- Also called **unbounded** or **galloping** search
- Used today inside several merge and set-intersection algorithms

Intuition — searching an unbounded list

- No known end? Grow a **guess** at the end instead
- Check index 1, then 2, then 4, then 8, then 16, ...
- Stop doubling once you have clearly gone too far
- Now a normal, **bounded** binary search finishes the job

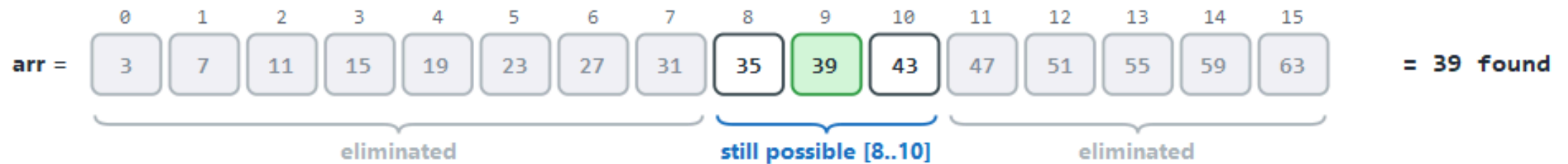
The exponential search idea

- Check `arr[0]` first — the target might be right there
- Grow `bound` : 1, 2, 4, 8, ... until `arr[bound] >= target`
- Run ordinary **binary search** inside `[bound/2, bound]`
- Needs a **sorted** array, same as every strategy so far

Exponential search, step by step

target = 39

comparisons: 7

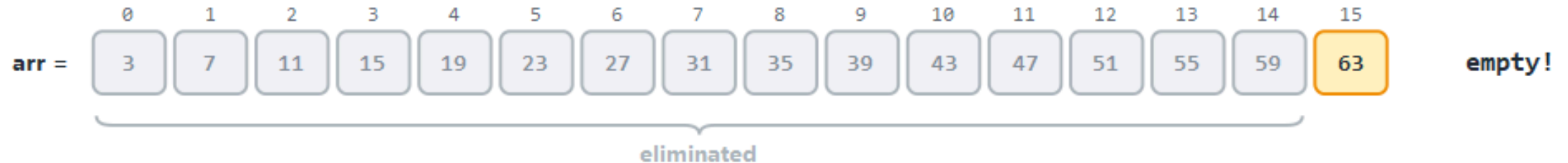


Result: `arr[9] = 39`, found in 7 comparisons. When the target is near the front, bound-finding is very cheap.

Edge case — target beyond the last value

target = 999

comparisons: 9



Not found: 999 is not in the array. 9 comparisons in total: bound-finding is $O(\log \text{index})$, then binary search is $O(\log(\text{bound}))$.

Code — growing the bound

```
int comp = 1;
if (arr[0] == target) return 0;
int bound = 1;
while (bound < n) {
    comp++;
    if (arr[bound] >= target) break;
    bound *= 2;
}
```

Code — binary search inside the bound

```
int lo = bound / 2;
int hi = (bound < n) ? bound : n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    comp++;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
```

Complexity

- Bound-finding phase: $O(\log \text{index})$, where `index` is where the target actually is
- Binary search phase: $O(\log(\text{bound}))$, so $O(\log \text{index})$ too
- Total: $O(\log \text{index})$ — much better than $O(\log n)$ when the target is near the front

Common mistakes

- Forgetting to clamp `bound` to `n - 1` near the end
- Restarting binary search from `[0, bound]` instead of `[bound/2, bound]` — wastes the doubling phase's work
- Assuming this needs a known `n` — it explicitly does not

Mini-quiz

An array has one million elements, but the target sits at index 3. Roughly how many comparisons does exponential search need?

Answer

Only a handful — about $\log_2(3) + 1 \approx 3$
comparisons total, regardless of the array
holding a million elements.

5. Fibonacci Search

A question to start

Every strategy so far divides: n/block ,
 $(\text{hi}-\text{lo})/2$, $\text{bound}/2$. What if the hardware
running your program could not divide cheaply?

A short history

- **1953** — Jack Kiefer's Fibonacci search **technique**, first developed for optimization, not arrays
- Adapted to array search soon after
- Uses only **addition and subtraction** — no **/**, no *****

Intuition — Fibonacci numbers as a ruler

- Fibonacci numbers: 1, 1, 2, 3, 5, 8, 13, 21, ...
- Each one is the sum of the two before it
- Pick the smallest Fibonacci number $\geq n$ as a "ruler"
- Split the range using that ruler, not by dividing

The Fibonacci search idea

- Requires a **sorted** array, exactly like binary search
- Track a triple `(fib, fib1, fib2)`, one Fibonacci step apart
- Probe at `offset + fib2`; compare, then shrink the triple
- `< target` : shrink by one step; `> target` : shrink by two

Fibonacci search, step by step

target = 39

fib=5, fib1=3, fib2=2, offset=7

comparisons: 3



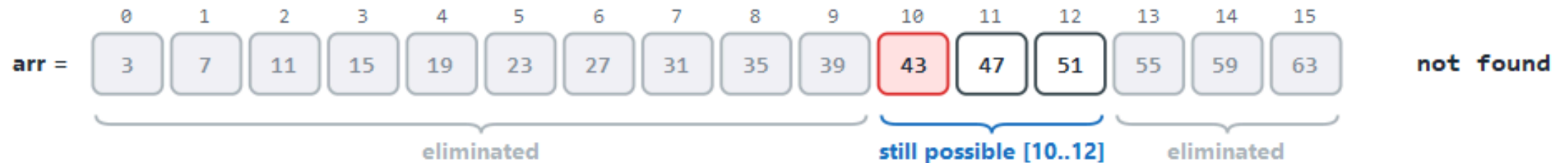
Result: `arr[9] = 39`, found in 3 comparisons. Fibonacci search uses only addition/subtraction — no multiplication or division.

Edge case — target in range but absent

target = 40

fib=1, fib1=1, fib2=0, offset=9

comparisons: 5



Not found: 40 is not in the array. 5 comparisons in total. Fibonacci search uses only addition/subtraction — no multiplication or division.

Code — the Fibonacci triple and probe

```
int fib2=0, fib1=1, fib=fib1+fib2;
while (fib < n) {
    fib2=fib1; fib1=fib; fib=fib1+fib2;
}
int offset = -1;
while (fib > 1) {
    int i = (offset+fib2 < n-1)
           ? offset+fib2 : n-1;
```

Code — shrinking the triple

```
if (arr[i] < target) {  
    fib=fib1; fib1=fib2; fib2=fib-fib1;  
    offset = i;  
} else if (arr[i] > target) {  
    fib=fib2; fib1=fib1-fib2; fib2=fib-fib1;  
} else {  
    return i;  
}
```

Complexity

- Same order as binary search: $O(\log n)$
- Slightly more comparisons in practice, on average
- Its real advantage: **no division or multiplication**
- Historically valuable on hardware without fast division

Common mistakes

- Forgetting the leftover single-element check at the end
- Computing the Fibonacci triple **inside** the main loop
- Assuming it beats binary search's $O(\log n)$ — it does not

Mini-quiz

Why might Fibonacci search have mattered more in the 1950s and 1960s than it does on today's hardware?

Answer

Division was expensive, or unavailable, on early hardware. Fibonacci search's pure addition/subtraction avoided that cost entirely.

6. Comparing the Five Search Strategies

Comparison — what each one needs

Strategy	Needs	Best case
Jump search	Sorted array	$O(\sqrt{n})$
Interpolation search	Sorted, uniform	$O(\log \log n)$
Exponential search	Sorted array	$O(\log \text{ index})$

Comparison — Fibonacci and the recap

Strategy	Needs	Best case
Fibonacci search	Sorted array	$O(\log n)$
Binary search (Week 1)	Sorted array	$O(\log n)$
Linear search (Week 1)	Nothing	$O(n)$

When to use which

- Data roughly uniform? **Interpolation** search
- Target usually near the front? **Exponential** search
- Simple code, no division available? **Jump** or **Fibonacci**
- No special structure? Plain **binary search** is enough

Mini-quiz

A sorted log file grows constantly, so its length `n` is never known in advance. Which of today's strategies fits best, and why?

Answer

Exponential search. It only grows a bound until it overshoots — it never needs n known in advance, unlike binary search.

7. Hashing: From Direct Addressing to Hash Functions

A question to start

Every search so far **compares** values, again and again. Could we instead **compute** a key's location directly, in one step?

A short history

- **1953** — Hans Peter Luhn, at IBM, proposes using a computed function to place records directly
- This internal memo is widely credited as hashing's origin
- The idea: turn a **key** into a **table index**, by formula

Intuition — direct addressing

- Keys are small integers, $0..m-1$? Just use the key itself as the array index — $O(1)$, no computing needed
- `table[key] = value` — direct addressing, no hash at all
- Problem: real keys rarely fit $0..m-1$ neatly
- Student IDs, phone numbers: far too many possible values

The hashing idea

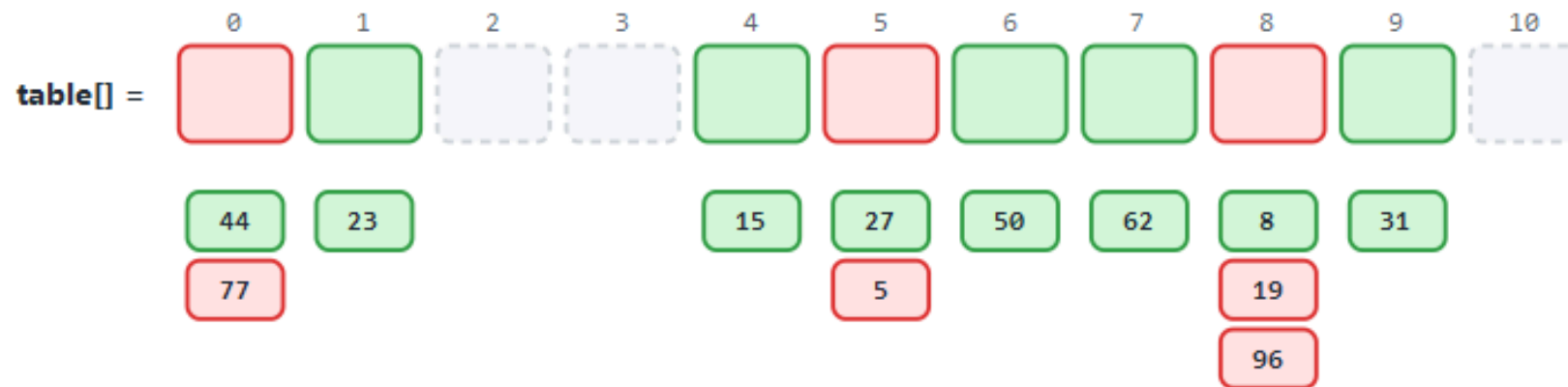
- A **hash function** $h(\text{key})$ maps any key to $0..m-1$
- m = the table size; h must be fast, ideally $O(1)$
- Two different keys landing on the same index: a **collision**
- Sections 8–10 build the machinery to handle collisions

The division method

- Simplest common hash function: $h(k) = k \bmod m$
- Maps any integer key into $[0, m-1]$ — always, in $O(1)$
- The **choice of m** changes how well keys spread out
- A careless m can make many keys collide on purpose

Division method, step by step

$m = 11$



$h(5) = 5 \bmod 11 = 5$

Done: 12 keys, 4 collisions, 8/11 cells used. The choice of m matters: an m with no common factor with the keys' pattern (usually prime) reduces clustering.

Edge case — m a power of 10: disaster

m = 10



Done: 11 keys, 10 collisions, 1/10 cells used. The choice of m matters: an m with no common factor with the keys' pattern (usually prime) reduces clustering.

Why prime m ?

- Keys sharing a **common factor** with m collide badly
- m a power of 10, keys multiples of 10: total collapse
- A **prime** m shares no factor with most key patterns
- Rule of thumb: pick m prime, avoid powers of 2 or 10

Code — the division hash function

```
/* + m) % m guards negative keys in C */  
int hash_division(int key, int m) {  
    return ((key % m) + m) % m;  
}
```

Complexity

- $h(k) = k \bmod m$: one division — **$O(1)$** , always
- This $O(1)$ cost is what makes hashing attractive at all
- The real cost of a hash table lives in **collisions**
- Sections 8–10 are entirely about managing that cost

Common mistakes

- Picking `m` as a power of 2 or 10 out of convenience
- Forgetting the negative-key guard in languages like C or Java
- Assuming $O(1)$ always holds, even with many collisions

Mini-quiz

$m = 8$ and every key in a dataset happens to be **even**. What fraction of the table's cells can ever be used?

Answer

At most half. An even key mod 8 can only land on an even index (0, 2, 4, 6) — the odd-indexed cells are never reachable at all.

8. Collisions and Separate Chaining

A question to start

Two different keys hash to the same index.
The table has only one cell there. Where
does the second key go?

Intuition — a shared mailbox slot

- Two students share the same mailbox number by chance
- Solution: hang a **small list** off that one mailbox slot
- Any number of keys can share one slot, one after another
- This is **separate chaining**: a linked list per bucket

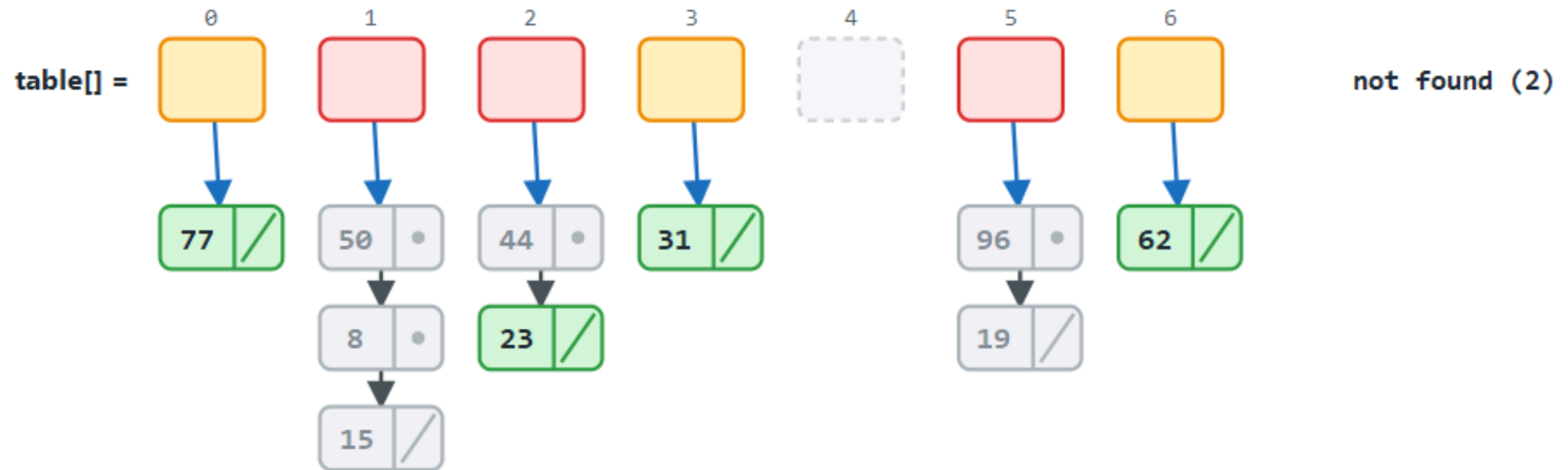
The chaining idea

- Table cell `i` holds the **head** of a linked list
- Insert: new node becomes the list's **head** — $O(1)$
- Search: walk the list, comparing each key — $O(\text{chain length})$
- Empty chain (`NULL`): the key is certainly not present

Separate chaining, step by step

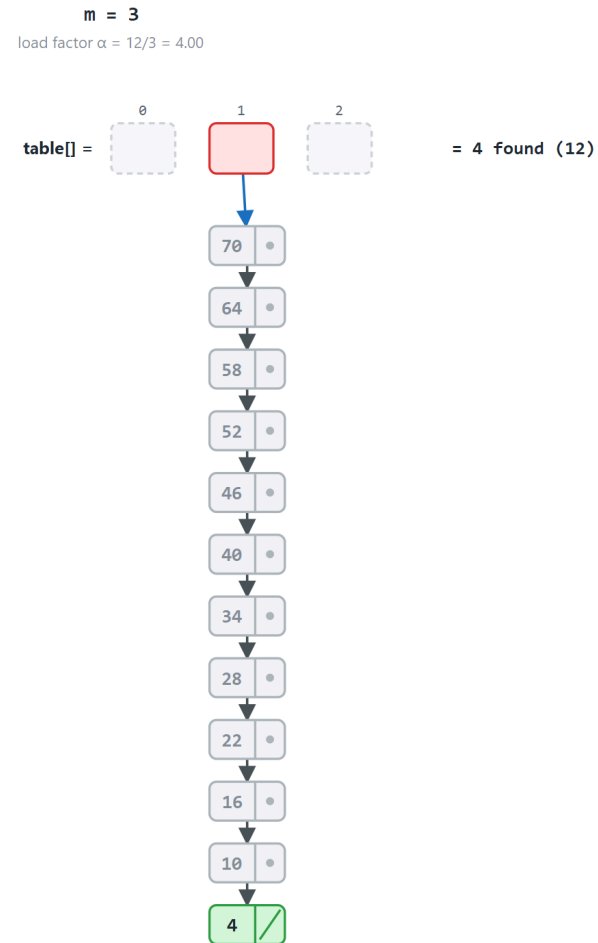
$$m = 7$$

$$\text{load factor } \alpha = 10/7 = 1.43$$



Done: 10 keys, $\alpha = 1.43$. 4 searches: 2 hits, 2 misses. Search cost depends on chain length, on average $O(1 + \alpha)$.

Edge case — a heavily loaded table



Done: 12 keys, $\alpha = 4.00$. 3 searches: 2 hits, 1 miss. Search cost depends on chain length, on average $O(1 + \alpha)$.

Code — insert (head of the chain)

```
void insert(int key) {  
    int idx = key % M;  
    Node *n = malloc(sizeof(Node));  
    n->key = key;  
    n->next = table[idx]; /* new head */  
    table[idx] = n;  
}
```

Code — search (walk the chain)

```
bool search(int key) {  
    int idx = key % M;  
    for (Node *c = table[idx]; c;  
        c = c->next) {  
        if (c->key == key) return true;  
    }  
    return false;  
}
```

Load factor

- $\alpha = n / m$ — keys stored, divided by table size
- $\alpha = 0.5$: chains average length 0.5 — nearly $O(1)$
- $\alpha = 4$: chains average length 4 — real cost, not $O(1)$
- A hash table's **speed** depends directly on α

Complexity

- Insert: always $O(1)$ — straight to the chain's head
- Search (average): $O(1 + \alpha)$ — one hash, plus the chain
- Search (worst case): $O(n)$ — every key in one chain
- Keeping α small is what keeps chaining fast in practice

Common mistakes

- Letting α grow unbounded without ever resizing the table
- Forgetting an empty chain (`NULL`) means "definitely absent"
- Confusing "collision" with "error" — collisions are expected

Mini-quiz

A table has $m = 5$ buckets and $n = 20$ keys, spread evenly by chaining. What is the load factor, and the average chain length?

Answer

$\alpha = 20 / 5 = 4$. On average, each chain holds about 4 keys — search costs roughly 4 comparisons, not $O(1)$.

9. Open Addressing

A question to start

Separate chaining needs extra memory for list nodes. Could every key instead live **directly inside** the table's own cells?

The open addressing idea

- No lists: every key lives **in** the table array itself
- Collision at $h(\text{key})$? **Probe** a sequence of other cells
- Stop at the first **empty** cell — the key settles there
- Deleting needs a **tombstone** marker, not a true empty cell
- Three probing rules follow: linear, quadratic, double hashing

9a. Linear Probing

The linear probing idea

- Collision at $h(\text{key})$? Try $h(\text{key})+1$, $+2$, $+3$, ...
- Wrap around with $\text{mod } m$ when the index passes the end
- Stop at the first cell that is empty **or** deleted
- Simple to code, but keys tend to pile up in **runs**

Linear probing, step by step

$m = 11$



empty cell – stop

$h = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 0$

Done: 10/10 inserts placed. Linear probing is simple but causes primary clustering: occupied runs grow, and growing runs attract even more collisions.

Edge case — search after a delete

`m = 11`



Done: 10/10 inserts placed. Linear probing is simple but causes primary clustering: occupied runs grow, and growing runs attract even more collisions.

Edge case — the table fills up

$m = 8$



table full!

$h = 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 0 \rightarrow 1 \rightarrow 2$

Done: 8/10 inserts placed (2 rejected, the table was full). Linear probing is simple but causes primary clustering: occupied runs grow, and growing runs attract even more collisions.

Code — insert with linear probing

```
bool insert(int key) {
    int idx = key % M;
    for (int i = 0; i < M; i++) {
        if (state[idx] != OCCUPIED) {
            table[idx] = key;
            state[idx] = OCCUPIED;
            return true;
        }
        idx = (idx + 1) % M;
    }
    return false; /* table full */
}
```

Code — delete leaves a tombstone

```
bool delete_key(int key) {  
    int idx = key % M;  
    for (int i = 0; i < M; i++) {  
        if (state[idx] == EMPTY)  
            return false;  
        if (table[idx] == key) {  
            state[idx] = DELETED;  
            return true;  
        }  
        idx = (idx + 1) % M;  
    }  
    return false;  
}
```

Primary clustering

- Colliding keys always retrace the **same** linear path
- Occupied runs grow, and growing runs attract more collisions
- This snowball effect is called **primary clustering**
- Sections 9b and 9c fix this with a less predictable step

Complexity

- Insert / search, low load factor: close to $O(1)$
- As α approaches 1: clustering makes cost climb sharply
- Worst case: $O(m)$ — scanning the whole table
- Keeping α well below 1 matters even more than chaining

Common mistakes

- Clearing a deleted cell to EMPTY instead of DELETED
- Letting α reach 1.0 — insertion then never terminates cleanly
- Forgetting search must also walk **past** DELETED cells

Mini-quiz

A key is deleted, leaving a tombstone. A later `search()` walks past that cell without stopping. Why is that the correct behavior?

Answer

Because a key placed after the deleted one may have probed right past it. Stopping at a tombstone would wrongly report it as absent.

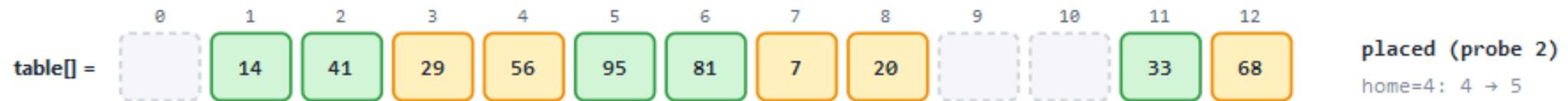
9b. Quadratic Probing

The quadratic probing idea

- Collision at `home` ? Try `home + 12` , `home + 22` , ...
- Step size grows fast: 1, 4, 9, 16, ... instead of 1, 1, 1, ...
- Spreads colliding keys apart — reduces primary clustering
- New trap: the `i2` sequence can **cycle** without success

Quadratic probing, step by step

$m = 13$



Done: 10/10 inserts placed. Quadratic probing reduces clustering but is sensitive to the choice of m .

Edge case — m a power of 2: a cycle

$m = 8$

table[] =

0	1	2	3	4	5	6	7
8	9	26	19	12	5	14	7

table full!

home=0: $0 \rightarrow 1 \rightarrow 4 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 4 \rightarrow 1$

Done: 8/10 inserts placed, 1 cycled despite free space existing (m not prime / $\alpha > 0.5$). Quadratic probing reduces clustering but is sensitive to the choice of m .

Code — insert with quadratic probing

```
bool insert(int key) {  
    int home = key % M;  
    for (int i = 0; i < M; i++) {  
        int idx = (home + i*i) % M;  
        if (state[idx] != OCCUPIED) {  
            table[idx] = key;  
            state[idx] = OCCUPIED;  
            return true;  
        }  
    }  
    return false;  
}
```

Complexity, and why m matters

- Insert / search, low load factor: close to $O(1)$
- Less clustering than linear probing at the same α
- Requires m **prime** and $\alpha \leq 0.5$ — else the i^2 sequence can **cycle** without reaching a free cell

Common mistakes

- Choosing m as a power of 2, "because it's a round number"
- Letting α exceed 0.5 — cycling becomes likely
- Treating a cycled insert the same as a genuinely full table

Mini-quiz

$m = 8$. Why is quadratic probing riskier here than with $m = 11$?

Answer

8 is a power of 2, not prime. The i^2 sequence can cycle through only a few cells, even when the table still has free space.

9c. Double Hashing

The double hashing idea

- Two hash functions: $h1(key)$ for the home, $h2(key)$ for the **step**
- $probe(i) = (h1(key) + i * h2(key)) \bmod m$
- Different keys usually get **different** steps
- Colliding keys no longer retrace the same path

Double hashing, step by step

m = 13, R = 11

$h1(95) = 4, h2(95) = 11 - (95 \bmod 11) = 4$

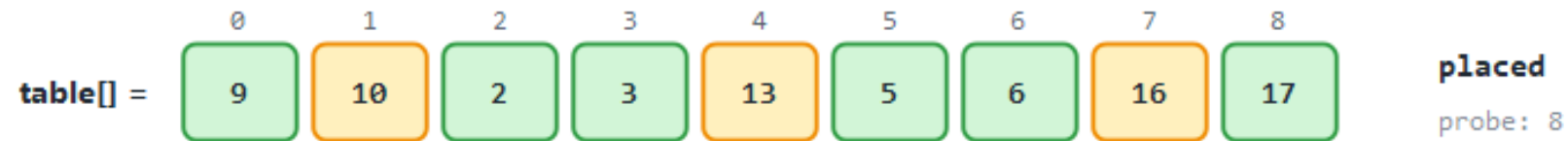


Done: 10/10 inserts, 15 probes in total. Linear probing on the very same keys places 10/10, taking 24 probes — double hashing usually means less clustering and fewer probes.

Edge case — a shared-factor cycle

m = 9, R = 7

$h_1(17) = 8, h_2(17) = 7 - (17 \bmod 7) = 4$



Done: 9/10 inserts, 18 probes in total. Linear probing on the very same keys places 9/10, taking 22 probes — double hashing usually means less clustering and fewer probes.

Code — the two hash functions

```
int h1(int key, int m) {  
    return key % m;  
}  
int h2(int key, int r) {  
    return r - (key % r); /* r < m, prime */  
}
```

Code — probing with two hashes

```
int idx = h1(key, M);
int step = h2(key, R);
for (int i = 0; i < M; i++) {
    if (state[idx] != OCCUPIED) {
        table[idx] = key;
        return idx;
    }
    idx = (idx + step) % M;
}
```

Complexity, and why m matters

- Insert / search: close to $O(1)$ for α well below 1
- Least clustering of the three probing rules
- Needs m prime so every step reaches **every** cell
- Usually the best open-addressing choice in practice

Common mistakes

- Letting `h2(key)` possibly return `0` — an infinite step
- Choosing `m` and `R` so they share a common factor
- Forgetting `R` itself must be prime, and `R < m`

Mini-quiz

Linear, quadratic, and double hashing all require m prime for good behavior. Which one is **most** forgiving if m is not prime?

Answer

Linear probing. It still reaches every cell whenever `m` is composite — only its **clustering** gets worse, not its correctness.

10. Rehashing

A question to start

Every technique in Section 9 gets slower as α climbs toward 1. What should happen before that actually occurs?

The rehashing idea

- Pick a **threshold** for α (commonly around 0.7–0.8)
- Crossing it triggers a **rehash**: build a bigger table
- New size: the next **prime** at least $2 * m$
- **Every** key must be reinserted — its index may change

Why every key must move

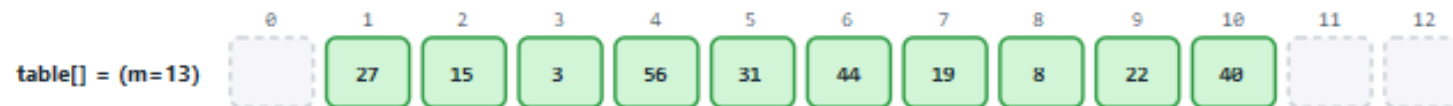
- $h(k) = k \bmod m$ — change m , and $h(k)$ usually changes
- Copying old cells to new ones directly would be wrong
- Instead: **reinsert** each key into the new, empty table
- This uses the exact same `insert()` already written

Rehashing, step by step

threshold = 0.8

$\alpha = 10/13 = 0.77$

insert(40) → idx 10



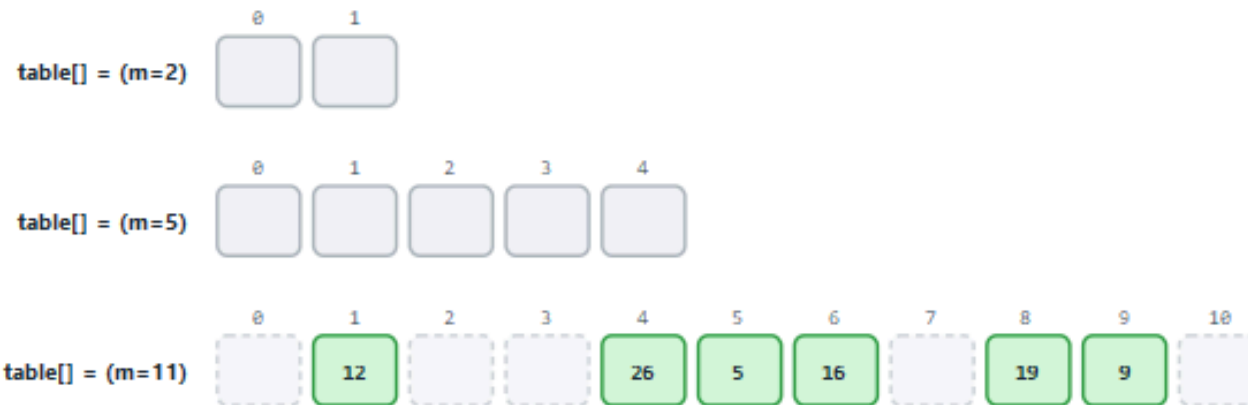
Done: 10 keys inserted, the table grew from $m_0 = 6$ to $m = 13$. Rehashing is expensive ($O(n)$) but rare — amortized insertion is still $O(1)$.

Edge case — growing twice in a row

threshold = 0.8

$\alpha = 6/11 = 0.55$

insert(16) → idx 6



Done: 6 keys inserted, the table grew from $m_0 = 2$ to $m = 11$. Rehashing is expensive ($O(n)$) but rare — amortized insertion is still $O(1)$.

Code — finding the next prime size

```
int is_prime(int x) {  
    if (x < 2) return 0;  
    for (int i=2; i*i<=x; i++)  
        if (x % i == 0) return 0;  
    return 1;  
}  
int next_prime(int x) {  
    while (!is_prime(x)) x++;  
    return x;  
}
```

Code — growing and reinserting

```
void rehash(void) {  
    int newM = next_prime(2 * m);  
    /* ... allocate newState, newTable ... */  
    for (int i = 0; i < m; i++)  
        if (state[i] == OCCUPIED)  
            insert_into(newState, newTable,  
                        newM, table[i]);  
    m = newM; /* ... swap in new arrays ... */  
}
```

Amortized complexity

- A single rehash costs $O(n)$ — every key reinserted
- Rehashes happen **rarely**: table size at least doubles
- Spread across many inserts, average cost stays $O(1)$
- This is the same **amortized** idea as Week 2's dynamic array

Common mistakes

- Never rehashing at all — α grows without bound
- Growing to a size that is not prime
- Rehashing on every single insert instead of past a threshold

Mini-quiz

A table with $m = 6$ holds $n = 5$ keys, and the threshold is 0.8 . One more key is inserted. Does a rehash trigger, and to what size?

Answer

Yes. $\alpha = 6/6 = 1.0 > 0.8$ — a rehash triggers. New size: `next_prime(12) = 13`.

11. Choosing a Collision-Resolution Technique

Chaining vs. open addressing

	Chaining	Open addressing
Extra memory	Yes, per node	No, in-place
High α	Degrades gracefully	Degrades sharply
Deletion	Simple	Needs tombstones

Choosing within open addressing

- Simple code, α kept low? **Linear** probing is enough
- Want less clustering, willing to require prime m ?
Quadratic probing
- Want the best spread available? **Double** hashing

A practical rule of thumb

- Deletions are frequent or memory is not tight? **Chaining**
- Memory matters and α is kept well below 1? **Open addressing**
- Either way: **rehash** before α gets too large
- No universally "best" choice — only best **for your case**

Mini-quiz

A hash table will hold millions of keys, with almost no deletions, and memory is tight. Which family fits best, and why?

Answer

Open addressing, likely **double hashing**.

No deletions to worry about, and no extra per-node memory for millions of chain links.

Summary — faster search on sorted data

Idea	Key fact
Jump search	$O(\sqrt{n})$, block = $\sqrt{n}$
Interpolation search	$O(\log \log n)$ on uniform data
Exponential / Fibonacci	Unbounded n / no division

Summary — hashing and collisions

Idea	Key fact
Hash function	$h(k) = k \bmod m$, $O(1)$, m prime
Chaining	Linked list per bucket, $O(1 + \alpha)$
Open addressing	In-place, needs tombstones, rehashing

The big picture

Sorted data supports faster-than-binary search by exploiting extra structure. Hashing drops comparison entirely: compute a location, handle collisions, and grow the table before it fills.

Self-check round

Four short questions. Think before the answer appears on the next slide. Full exercises and a ten-question quiz are in the week notes.

1. An array has 400 elements. What block size does jump search use, and roughly how many comparisons in the worst case?

$\text{block} = \text{sqrt}(400) = 20$. Worst case: about $20 + 20 = 40$ comparisons — far below linear search's 400.

2. Why does interpolation search's formula need a guard against `arr[hi] == arr[lo]`?

Without it, the formula's denominator would be zero — a division-by-zero error, since an all-equal range makes

$$\text{arr}[\text{hi}] - \text{arr}[\text{lo}] = 0.$$

3. Why should a hash table's size m usually be chosen prime?

A prime m shares no common factor with most key patterns, avoiding the severe clustering a composite m (like a power of 10) can cause.

4. Why must a deleted cell in open addressing become a tombstone, not a plain empty cell?

A later search stops at the first truly empty cell. Clearing to empty would wrongly cut off the probe chain for keys placed after it.

Next week

Week 7 — Project Demonstrations

No new algorithms this week: teams present their project's search or hashing component, followed by Week 8's quiz before graph algorithms resume in Week 9.

References (1/2)

- Course syllabus, Week 6: `docs/syllabus/syllabus.en.md`
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed. MIT Press
- Knuth. *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed
- Bentley, J., Yao, A. (1976). "An almost optimal algorithm for unbounded searching"
- Luhn, H. P. (1953). Internal IBM memorandum on hashing

References (2/2)

- Kiefer, J. (1953). "Sequential minimax search for a maximum"
- Peterson, W. W. (1957). "Addressing for random-access storage"
- Sedgewick, Wayne. *Algorithms*, 4th ed. Addison-Wesley
- [williamfiset/Algorithms](#) · Programiz DSA